2. Build

All building goes through `v001/build.sh`; the GUI's Build page, `verify.sh` and `bootstrap.sh` all call it. Run everything from `jac317/v001`.

2.1 What gets built

Program	What it is
`treesync`	The GUI
`treesync-cli`	The command-line program; does every sync (the GUI starts it)
`treesync-test`	The core tests (built with `--test`)

Each compiler has its own folder: `out/gcc/` and `out/clang/`. The two builds are the same program; TreeSync is also a proving ground for GCC 15 and Clang 21 with C++26.

2.2 build.sh

```
./build.sh --compiler gcc                # debug build with gcc 15
./build.sh --compiler clang              # debug build with clang 21 (the default compiler)
./build.sh --compiler gcc --release      # optimized build: use this for real syncs
./build.sh --compiler gcc --test         # build, then run the core tests
./build.sh --compiler gcc --release --test
./build.sh --compiler gcc --clean        # compile everything, not only what changed
```

Option	What it does
`--compiler gcc` / `clang`	Which compiler (default clang)
`--release`	Optimized build (default: debug)
`--test`	Also build `treesync-test` and run it; the exit code is non-zero if a test fails
`--clean`	Rebuild everything (fair build times when comparing compilers)
`--tsan`	Instead of the build: the tests under ThreadSanitizer (clang), into `out/clang-tsan/`
`--asan`	The tests under AddressSanitizer + UBSan, then `treesync-cli --bench` under them
`--valgrind`	The tests under valgrind memcheck (slow)
`--helgrind`	The tests under valgrind helgrind (very slow)

Debug or release? Debug builds check more and are easier to debug, but are much slower: on many small files about 4 times the CPU. Always run real syncs with a **release** build.

`build.sh` remembers which type is in each `out/` folder and links the programs again when you switch, so a debug build never leaves a release program behind or the other way round.

A test run ends with a line such as `passed: 290/290 checks`. A debug build skips one test (a 4.5 GiB sparse copy, too slow without optimization) and says so; a release test run has all of them (`passed: 303/303 checks`).

2.3 The Build page in the GUI

File > Build... opens it, and it is also the first screen when TreeSync starts (untick **Show this page when TreeSync starts** to go straight to the main window).

- **Compiler**: gcc, clang, or both one after the other.
- **Type**: Debug or Release.

- **Run the tests (treesync-test) after building**: the one switch for testing. Unticked, **Build** only builds. Ticked, the tests run, plus the checked runs below it:

    - Also under ThreadSanitizer (clang; data races)
    - Also under AddressSanitizer + UBSan (memory errors, undefined behaviour)
    - Also under valgrind memcheck (slow)
    - Also under valgrind helgrind (very slow)

  These keep their ticks while "Run the tests" is off; they are greyed out and skipped. - **Clean build**: compile everything. - **Compare the compilers**: versions, C++26 features, `import std`, speed and size, side by side. - **Display**: Wayland (recommended) or X11. - **This machine**: whether the requirements are present (**Check again**, **Open terminal to fix...**).

The command lines it will run are shown at the bottom. **Build** runs them one after another and shows each one's output; the results screen sums them up. **Continue to TreeSync using GCC exe / Clang exe** starts the main window with that build.

Your choices are saved in `~/.config/treesync/build`.

2.4 The sanitizer and valgrind runs

These find bugs the normal tests cannot see: data races between the copy threads, memory errors, undefined behaviour. They take much longer than the normal tests, so run them when threading or memory handling changed:

```
./build.sh --compiler clang --tsan
./build.sh --compiler gcc --asan
./build.sh --compiler gcc --valgrind
./build.sh --compiler gcc --helgrind
```

They need the optional `checks` requirement (see 1.4). Each builds into its own folder (`out/gcc-asan/` and so on) and never touches the programs in `out/gcc/`.

2.5 verify.sh: the full check, and the pre-push hook

```
./verify.sh                 # gcc and clang
./verify.sh --compiler gcc  # one compiler
```

For each compiler it does a **debug build with every test**, then a **release build with every test**. It stops at the first failure and ends with **release builds in `out/`**, the programs the GUI runs. Each step's full output is in `out/verify/<compiler>-<type>.log`. It also checks that each step really tested the type it claims. A run takes well under a minute when little has changed:

```
verify: debug, then release; every test; compilers: gcc clang
 gcc    debug    building and testing ... ok    10 s  passed: 296/296 checks
 clang  debug    building and testing ... ok     7 s  passed: 296/296 checks
 gcc    release  building and testing ... ok    13 s  passed: 303/303 checks
 clang  release  building and testing ... ok    13 s  passed: 303/303 checks
 took 43 s
 verify: all good; out/ holds release builds
```

The pre-push hook. `v001/setup/hooks/pre-push` runs `verify.sh` before every `git push`; a failure stops the push. `bootstrap.sh` sets it up; by hand:

```
git config core.hooksPath v001/setup/hooks
```

To push once without the check: `git push --no-verify`.

2.6 Install for root

You build as yourself; most syncs need root. Root must not run programs your user can change (anything running as you could swap them first), so the release programs are **installed** into the usual system folders, owned by root:

```
./install.sh                   # the gcc release build; asks for your sudo password
./install.sh --compiler clang  # the clang one
./install.sh --dry-run         # only say what it would do
./install.sh --uninstall       # remove it all again
```

Installed	Where
The programs	`/usr/local/bin/treesync`, `/usr/local/bin/treesync-cli` (`root:root`, mode 755, like every program in `/usr/bin`)
The manual	`/usr/local/share/doc/treesync/pdf/` (the Help menu finds it there)
What was installed	`/usr/local/share/doc/treesync/INSTALLED`: compiler, commit, who, when, the SHA-256 of `treesync-cli`
Desktop menu entry	`/usr/local/share/applications/treesync.desktop`: **TreeSync** in the app list
polkit action	`/usr/share/polkit-1/actions/org.treesync.run-as-root.policy`: the GUI's root runs ask for the password once, then not again for a few minutes

It refuses a debug build: run `./verify.sh` (or `./build.sh --compiler gcc --release`) first. Programs go in under a temporary name and are renamed into place, so a TreeSync that is running keeps its copy. In the GUI, **File > Install for root...** does the same through `pkexec`.

Run it again after every build you want to use as root. The installed copy does not follow `out/` by itself.

The installed copy only runs. It has no Build page and no Build menu: there is no source next to it. It syncs as root for administrators, and as themselves for anyone else (see 3.9).

2.7 Building as root

You do not need to, and the GUI offers Build only to a normal user in the source tree (not to root, nor in the installed copy). The scripts still work as root, through `sudo`, `pkexec` or a root login: they use the U++ of the user the TreeSync folder belongs to (root's home has none), and give everything they made in `out/` back to that user, so your own next build can overwrite it.

2.8 After changing code

1. Build and test while working: `./build.sh --compiler gcc --test`.
2. Before pushing (or let the hook do it): `./verify.sh`.
3. When threads, locks or memory handling changed, also the sanitizer runs (2.4).

`verify.sh` leaves release builds in `out/`, so the GUI is fast again afterwards. 4. To use the new build as root: `./install.sh`.